Universitatea „Alexandru Ioan Cuza” din Iași

Facultatea de Informatică

Tehnici de Programare Multiprocesor

TEMA TPM

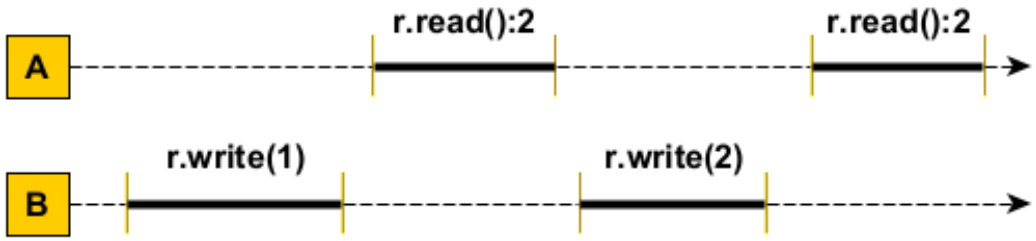
Motricală Alin-Gabriel Grupa 3A4

Gabriela Țibuleac Grupa 3E3

Seria A

Iași, 2026

Exercițiul 1



a) Linearizabilitate

Istoria **nu este linearizabilă**.

Pentru ca o istorie să fie linearizabilă, fiecare operație trebuie să aibă un punct de linearizare între începutul și sfârșitul execuției sale. În plus, trebuie respectată ordinea temporală a operațiilor care nu se suprapun.

În diagrama dată, threadul B execută mai întâi operația $r.write(1)$, după care threadul A execută o citire care returnează valoarea 2.

Observăm că prima operație $r.read()$ se termină înainte să înceapă $r.write(2)$.

Astfel, prima citire nu poate returna valoarea 2, deoarece această valoare nu a fost încă scrisă în registru.

Valoarea inițială a registrului este 0, iar singura scriere finalizată înaintea primei citiri este $r.write(1)$.

Prin urmare, prima citire ar trebui să returneze valoarea 1.

Nu există un punct de linearizare valid care să justifice prima citire cu valoarea 2, fără să încălcăm ordinea temporală reală.

În concluzie, **istoria nu este linearizabilă**.

b) Consistență secvențială

Istoria **este consistentă secvențial**.

Consistența secvențială presupune că putem găsi o ordine secvențială a tuturor operațiilor, astfel încât să respectăm ordinea operațiilor din fiecare thread și să obținem aceleași rezultate.

Spre deosebire de linearizabilitate, nu trebuie păstrată ordinea temporală dintre threaduri diferite.

În cazul nostru, threadul B execută două scrieri, în ordinea $r.write(1)$ și $r.write(2)$.

Threadul A execută două citiri, ambele returnând valoarea 2.

Putem construi următoarea ordine:

$$
\begin{aligned}
&B : r.write(1) \\
&\downarrow \\
&B : r.write(2) \\
&\downarrow \\
&A : r.read() = 2 \\
&\downarrow \\
&A : r.read() = 2
\end{aligned}
$$

Această ordine este validă deoarece:

- Threadul B păstrează ordinea celor două scrieri.
- Threadul A păstrează ordinea celor două citiri.
- Ambele citiri returnează 2, deoarece sunt plasate după operația $r.write(2)$.

Chiar dacă în execuția reală prima citire se termină înainte de începerea celei de-a doua scrieri, consistența secvențială permite această reordonare între threaduri.

Prin urmare, **istoria este consistentă secvențial**.

Exercițiul 2

În implementarea dată există câteva probleme legate de inițializarea threadului, vizibilitatea variabilelor și utilizarea lock-ului.

Probleme identificate

1. **Constructorul este declarat greșit.**

   Metoda `MyThread` are tipul de returnare `void`, deci nu este un constructor. Pentru a inițializa corect obiectul, trebuie eliminat `void`.

2. **Variabila alive nu este volatile.**

   Dacă variabila `alive` este modificată de un alt thread, nu avem garanția că modificarea va fi observată imediat. Declararea ei ca `volatile` asigură vizibilitatea modificărilor între threaduri.

3. **Folosirea incorectă a blocului try-finally.**

   În codul inițial, `unlock()` este apelat în blocul `finally`, chiar dacă threadul nu a reușit să obțină lock-ul.

   De exemplu, dacă `someNonExclusiveMethod()` aruncă o excepție, se ajunge la `finally` fără ca `lock()` să fi fost apelat. Același lucru se poate întâmpla dacă `lock()` eșuează înainte de obținerea lock-ului.

   De aceea, metoda neexclusivă se execută în afara secțiunii critice, iar apelul `lock()` este mutat înainte de `try`. Blocul `finally` se ocupă astfel doar de eliberarea unui lock obținut cu succes.

Varianta corectată

```
public class MyThread extends Thread {
    private final SharedObject sharedObject;
    private final SharedLock sharedLock;
    private volatile boolean alive = true;

    public MyThread(SharedObject sharedObject,
                    SharedLock sharedLock) {
        this.sharedObject = sharedObject;
        this.sharedLock = sharedLock;
    }

    public void stopRunning() {
        alive = false;
    }

    @Override
    public void run() {
        while (alive) {
```

```
                sharedObject.someNonExclusiveMethod();

                sharedLock.lock();
                try {
                    sharedObject.someExclusiveMethod();
                } finally {
                    sharedLock.unlock();
                }
            }
        }
}
```

Explicația soluției

În varianta corectată, constructorul inițializează obiectele partajate, iar variabila `alive` este declarată `volatile`, pentru a permite oprirea threadului din exterior.

Metoda `someNonExclusiveMethod()` se execută fără lock, deoarece nu necesită excludere mutuală.

Înainte de executarea metodei `someExclusiveMethod()`, threadul obține lock-ul prin `lock()`.

După obținerea lock-ului, folosim `try-finally` pentru a garanta eliberarea acestuia chiar dacă apare o excepție în secțiunea critică.

Astfel, apelul `unlock()` se execută doar după ce lock-ul a fost obținut cu succes.

Exercițiul 3

a) ShadyLock – Starvation-free

Algoritmul **nu este starvation-free**.

Un algoritm este starvation-free dacă fiecare thread care încearcă să obțină lock-ul reușește în cele din urmă să intre în secțiunea critică, presupunând că threadurile continuă să execute instrucțiuni și că secțiunile critice se termină.

În algoritmul dat, variabilele `turn` și `used` sunt partajate între threaduri. Problema este că instrucțiunile care verifică și modifică aceste variabile nu formează o operație atomică.

Putem construi o execuție cu două threaduri, T1 și T2, având ID-urile 1 și 2. Inițial, `used = false`.

Pas	Thread	Operație
1	T1	turn = 1
2	T1	verifică used = false
3	T2	turn = 2
4	T2	verifică used = false
5	T1	used = true
6	T1	turn != 1, reia bucla
7	T1	turn = 1
8	T1	verifică used = true
9	T2	used = true
10	T2	turn != 2, reia bucla

După această execuție, variabila `used` rămâne `true`, dar niciun thread nu a reușit să termine metoda `lock()`.

Ambele threaduri rămân în bucla interioară:

```
do {
    turn = me;
} while (used);
```

Variabila `used` poate deveni `false` doar prin apelul `unlock()`, însă niciun thread nu a obținut lock-ul și nu poate ajunge să îl elibereze.

Prin urmare, algoritmul poate ajunge într-o situație de deadlock. Aceasta încalcă și proprietatea starvation-free, deoarece threadurile pot aștepta la nesfârșit.

Concluzie: Algoritmul ShadyLock nu este starvation-free.

b) VeryShadyLock – Excludere mutuală

Algoritmul **nu asigură excluderea mutuală**.

Excluderea mutuală presupune că, la un moment dat, cel mult un thread poate executa secțiunea critică protejată de lock.

Deși algoritmul folosește un lock intern care garantează excluderea mutuală, acesta nu este obținut de toate threadurile.

Mai exact, dacă expresia `x != me` este falsă, threadul poate termina metoda `lock()` fără să obțină lock-ul intern.

Pentru a demonstra problema, considerăm două threaduri, T1 și T2, cu ID-urile 1 și 2. Inițial, `x = 0` și `y = 0`.

Pas	Thread	Operație
1	T1	x = 1
2	T1	y == 0, continuă
3	T2	x = 2
4	T2	y == 0, continuă
5	T2	y = 2
6	T2	x == 2, nu ia lock-ul intern
7	T2	Intră în secțiunea critică
8	T1	y = 1
9	T1	x != 1
10	T1	Obține lock-ul intern
11	T1	Intră în secțiunea critică

La pasul 7, T2 intră în secțiunea critică fără să obțină lock-ul intern.

Ulterior, T1 observă că `x != 1`, deci apelează `lock.lock()`.

Deoarece T2 nu a obținut lock-ul intern, acesta este liber, iar T1 îl poate obține imediat.

Astfel, T1 intră și el în secțiunea critică, în timp ce T2 se află încă acolo.

Acest lucru este posibil deoarece verificarea și modificarea variabilei `y` nu se realizează atomic. Ambele threaduri pot observa `y = 0` și pot continua.

Concluzie: Algoritmul VeryShadyLock nu garantează excluderea mutuală, chiar dacă lock-ul intern este corect.

c) ShadyChoice

Trebuie să demonstrăm că, într-o execuție cu $n > 1$ threaduri:

- cel mult un thread poate returna valoarea `red`;
- cel mult $n - 1$ threaduri pot returna valoarea `black`.

1. Cel mult un thread returnează red

Un thread poate returna `red` numai dacă citește `getWhite = false`, execută `getWhite = true` și apoi găsește `last == me`.

Presupunem, prin reducere la absurd, că două threaduri diferite, T1 și T2, returnează ambele `red`.

Notăm cu $W_i(last)$ scrierea ID-ului threadului Ti în variabila `last`, iar cu $R_i(last)$ citirea finală a acestei variabile. În mod asemănător, folosim $R_i(getWhite)$ și $W_i(getWhite)$ pentru accesările variabilei `getWhite`.

Pentru ca ambele threaduri să ajungă la `red`, fiecare trebuie să citească `getWhite = false` înainte ca celălalt să scrie `getWhite = true`. Deoarece scrierea în `last` se face înaintea acestei citiri, avem:

$$W_1(last) < R_1(getWhite) < W_2(getWhite) < R_2(last)$$

și, simetric:

$$W_2(last) < R_2(getWhite) < W_1(getWhite) < R_1(last)$$

Așadar, scrierile ambelor threaduri în `last` au loc înaintea citirilor finale ale ambelor threaduri.

Dacă T1 scrie ultimul dintre cele două, valoarea 2 a lui T2 este suprascrisă și T2 nu mai poate citi propriul ID. Dacă T2 scrie ultimul, T1 nu mai poate citi valoarea 1. Alte threaduri pot scrie alte ID-uri, dar nu pot face ca ambele threaduri să-și citească propriul ID, deoarece ID-urile sunt unice.

În ambele situații, cel puțin unul dintre threaduri nu poate returna `red`, contrar presupunerii inițiale.

Prin urmare, **cel mult un thread poate returna red**.

$$\boxed{\text{Numărul de red} \leq 1}$$

2. Cel mult n-1 threaduri returnează black

Pentru această proprietate, considerăm threadul T care execută ultimul instrucțiunea:

```
last = me;
```

Deoarece T este ultimul thread care modifică variabila `last`, valoarea acesteia va rămâne egală cu ID-ul lui T.

Există două posibilități:

1. Dacă T găsește `getWhite = true`, returnează imediat `white`.
2. Dacă T găsește `getWhite = false`, execută `getWhite = true`.

   La verificarea finală, `last == me` este adevărat, deoarece niciun alt thread nu mai modifică `last`.

   Prin urmare, T returnează `red`.

În ambele cazuri, T nu poate returna `black`.

Așadar, dintre cele n threaduri, cel puțin unul returnează `white` sau `red`.

Rezultă că cel mult $n - 1$ threaduri pot returna `black`.

$$\boxed{\text{Numărul de black} \leq n - 1}$$

Concluzie

Algoritmul ShadyChoice respectă ambele proprietăți cerute:

- maximum un thread poate returna `red`;
- maximum $n - 1$ threaduri pot returna `black`.

Exercițiul 4

Testele au fost efectuate pe următorul sistem:

- CPU: AMD Ryzen AI 7 350
- RAM: 32GB DDR5
- Sistem de operare: Windows 11
- Limbaj: Java

Rezultatele obținute

În tabelul următor sunt prezentate mediile timpilor de execuție obținuți din cele 5 rulări pentru fiecare configurație.

Algoritm	**Threaduri**	**Timp mediu (ms)**
Anderson	4	46,2307
Anderson	8	48,2609
Anderson	12	68,6965
CLH	4	20,3117
CLH	8	19,6496
CLH	12	28,8440

Analiza rezultatelor

Din rezultatele obținute observăm că algoritmul CLH a avut timpi de execuție mai mici decât Anderson în toate cele trei configurații.

Pentru 4 threaduri, Anderson a obținut un timp mediu de aproximativ 46,23 ms, în timp ce CLH a obținut 20,31 ms.

La 8 threaduri, Anderson a avut 48,26 ms, iar CLH 19,65 ms.

Pentru 12 threaduri, timpul mediu a crescut la 68,70 ms pentru Anderson și 28,84 ms pentru CLH.

O posibilă explicație pentru diferența de performanță este modul în care threadurile așteaptă eliberarea lock-ului.

În Anderson Lock, threadurile folosesc poziții dintr-un vector comun de indicatori. În CLH Lock, fiecare thread așteaptă pe indicatorul nodului predecesorului său.

Acest mod de organizare poate reduce traficul de memorie și costurile de sincronizare, în funcție de arhitectura procesorului și de nivelul de concurență.

Observăm și că, pentru 12 threaduri, timpii cresc la ambii algoritmi. Acest lucru poate fi explicat prin creșterea concurenței pentru lock și prin costurile de planificare a threadurilor.

Exercițiul 5

Pentru o paralelizare directă a algoritmului Dijkstra bidirecțional, considerăm că **2 threaduri** reprezintă o alegere eficientă, câte unul pentru fiecare direcție.

Algoritmul execută o căutare de la sursă la destinație (forward) și una de la destinație la sursă (backward). Într-un graf orientat, căutarea backward folosește muchiile inversate.

Fiecare thread are propriile structuri de date:

- o coadă de priorități, Q_f sau Q_b;
- distanțele calculate, d_f sau d_b;
- nodurile finalizate, S_f sau S_b.

Prin urmare, cele două căutări pot avansa în paralel, fără să modifice aceeași coadă. Totuși, există câteva informații care trebuie sincronizate: nodurile și distanțele finalizate de cealaltă căutare, minimul fiecărei cozi și valoarea comună μ.

Inițial, $\mu = \infty$. Aceasta reprezintă lungimea celui mai bun drum complet găsit până la momentul curent. Dacă un thread găsește un drum mai scurt, actualizarea lui μ trebuie făcută atomic sau sub protecția unui lock.

Condiția de oprire

Nu este corect să oprim algoritmul doar pentru că frontierele celor două căutări s-au întâlnit. Condiția de oprire este:

$$\boxed{\min(Q_f) + \min(Q_b) \geq \mu}$$

Aici, $\min(Q_f)$ și $\min(Q_b)$ sunt prioritățile minime ale celor două cozi. Dacă suma lor este cel puțin μ, nu mai putem îmbunătăți drumul găsit.

În implementarea paralelă, verificarea trebuie făcută pe o stare sincronizată. Nu putem ignora nodurile extrase deja din cozi, dar aflate încă în procesare. De exemplu, putem verifica oprirea după o sincronizare a threadurilor, când ambele au terminat pașii de relaxare începuți.

De ce nu mai mult de 2 threaduri?

Există doar două căutări principale, iar fiecare trebuie să selecteze nodul cu distanța minimă din coada sa. Împărțirea unei singure căutări între mai multe threaduri ar introduce sincronizare și dependențe suplimentare, fără o creștere garantată a performanței.

Concluzie: Pentru varianta bidirecțională obișnuită, am folosi **2 threaduri**, unul forward și unul backward. Pentru grafuri mici, costul sincronizării poate face însă varianta secvențială mai rapidă.